ACCORD PROJECT

Google Summer of Code 2026

Project: Template Logic Support for Playground

Project Mentors: Dan Selman, Sanket Shevkar

About Me

Name: Enosh Joseph P A
Gender: Male
Github: enoshjoseph
Linkedin: enosh-joseph-pa
Email: enoshjoseph72@gmail.com
Alternative Mail: enoshjoseph1@gmail.com
Discord: Enosh
Contact Number: +91 - 9446217841
Country: India
Timezone: IST (UTC +5:30)

PROFILE SUMMARY

Intro

I am Enosh Joseph P A, currently pursuing a Master of Computer Applications (MCA) at Mar Athanasius College of Engineering, Kerala, India, and I expect to graduate in 2026.
I was introduced to programming during the first year of my bachelor's studies, where I developed a strong interest in software development. This interest grew significantly as I explored web development and began building projects using HTML, CSS, and JavaScript.
As I gained experience, I expanded my skill set to include modern frontend technologies like React, along with backend development using Node.js, Express, and Python. More recently, I have also explored AI-based applications and built practical tools such as Chrome

extensions, which strengthened my understanding of real-world problem solving and system design.
I am particularly interested in building scalable, user-focused applications and contributing to open-source projects that have meaningful real-world impact.

Education

Degree: Masters in Computer Applications(2 year program)
Branch: Computer Applications
Current year: 2nd

Expected Graduation year: 2026
Institute: Mar Athanasius College of Engineering, Kerala, India.

My Skills

Programming languages	Python, JavaScript, TypeScript
Frontend Development	React.js, HTML, CSS, Tailwind CSS
Backend Development	Node.js, Express.js, REST APIs
Tools & Technologies:	Git,GitHub, Vercel, VS code, Postman

Contributions

I have started contributing to Accord Project ecosystem by exploring Template Playground and related repositories

Repository	Contribution	Pull Request	Status
template-playground	Documentation improvement PR	#517	Open

In addition to this, I actively engaged with the community through Discord discussions to better understand the project requirements and architecture.

To further strengthen my understanding, I built a working prototype that executes contract logic using TemplateArchiveProcessor.trigger().

This allowed me to validate the request-response execution pipeline and gain hands-on experience with template logic execution.

Through these efforts, I have developed a solid understanding of the Accord Project architecture, including TemplateMark processing, Concerto models, and contract execution workflows.

Prototype

To validate my understanding of template logic execution, I implemented a working prototype using TemplateArchiveProcessor.trigger().

The prototype successfully executes contract logic by passing a request object and receiving a structured response, including computed values and state changes. This demonstrates how logic execution can be integrated into the Template Playground.

Prototype Repository

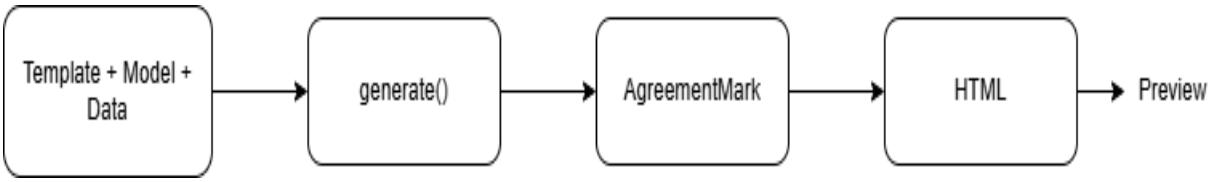
PROJECT DETAILS

The Accord Project is an open-source framework for building and executing smart legal contracts using structured templates, Concerto models, and executable logic.
The template playground is a web based interface that allows user to load templates, provide sample data and visualize contract output. It currently supports parsing TemplateMark and generating AgreementMark, which is rendered as HTML for preview.
The Playground follows a pipeline where templates are processed and transformed into a visual representation, enabling users to understand contract structure and content. However, it does not currently support executing the underlying contract logic defined in logic.ts.

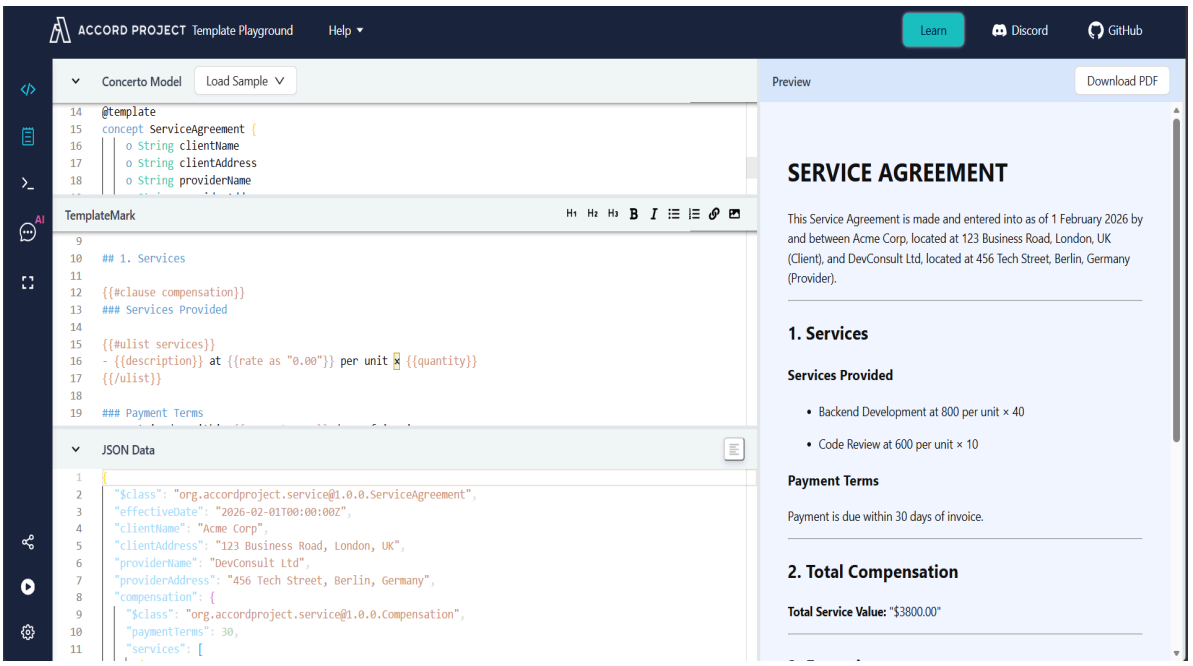
PROBLEM STATEMENT

While the Template Playground provides a useful interface for visualizing contract templates, it is currently limited to static rendering of TemplateMark into HTML.

It does not support execution of template logic, which is a core component of smart contracts in the Accord Project. As a result, user cannot simulate real contract behavior by providing input data and observing computed outputs such as responses, events or state updates



The Playground generates a static HTML preview of the contract based on the provided template and data. While this allows users to visualize the contract structure, it does not execute any underlying logic.



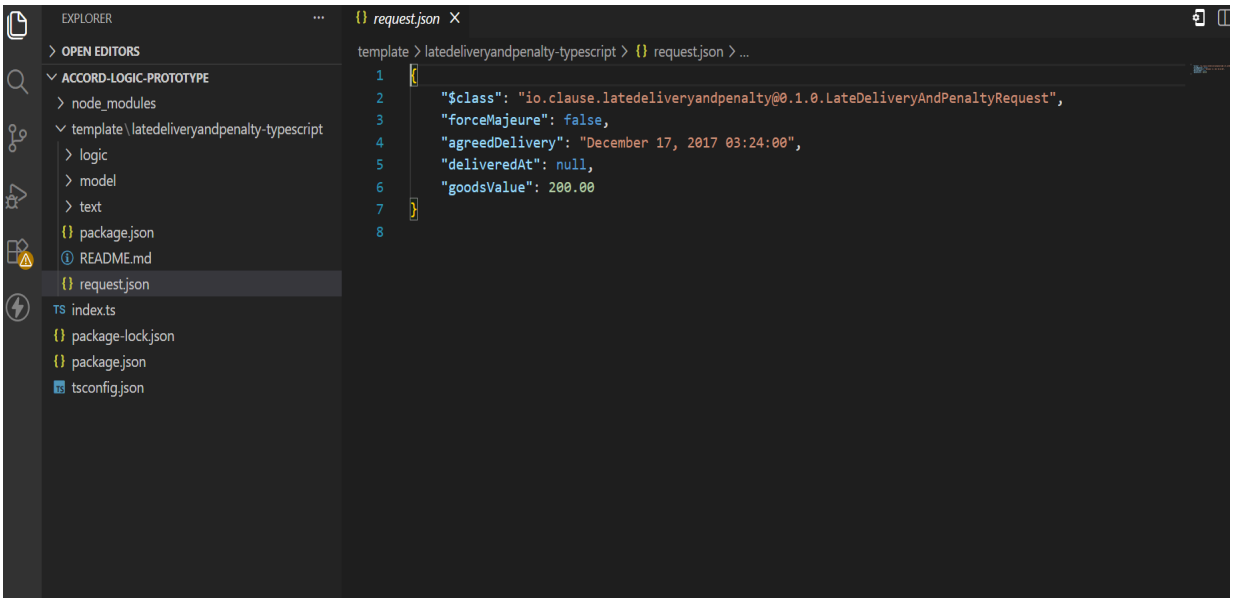
The left panel represents the user input, including the Concerto model, TemplateMark template, and JSON data. The right panel shows the generated HTML preview of the contract.

Currently, the system only renders static HTML output from the provided inputs and does not execute any template logic.

This highlights that the current system only supports rendering, and lacks the ability to process dynamic inputs and produce executable contract outputs.

To validate this gap, Implemented a prototype using TemplateArchiveProcessor.trigger(), where a structured request input produces a corresponding response output, demonstrating the execution pipeline that is currently missing in the playground.

Fig: template/latedeliveryandpenalty-typescript/request.json




```
===== RESPONSE =====
{
  "result": {
    "penalty": 5250,
    "buyerMayTerminate": true,
    "$timestamp": "2026-03-21T09:08:58.291Z",
    "$class": "io.clause.latedeliveryandpenalty@0.1.0.LateDeliveryAndPenaltyResponse"
  }
}
```

This clearly demonstrates the absence of a logic execution layer in the current Playground and the need for integrating trigger-based execution capabilities.

CURRENT CHALLENGES

The implementation of template logic execution within the Playground introduces several technical challenges due to the browser-based environment and architectural constraints.

- **Browser-based execution limitations:** The Playground runs completely in the browser. This makes it hard to support Node.js-specific modules and external dependencies that are often used in contract logic.
- **TypeScript compilation and execution:** Contract logic written in TypeScript must be compiled in the browser using twoslash. It also needs to run safely, which makes the execution process more complicated.
- **Handling unsupported dependencies:** Templates that depend on external libraries or imports may fail when they run. This requires proper validation and clear error handling for users.
- **Lack of persistent state:** While stateful behavior can be simulated within a session, there is no backend persistence, and all state is lost on refresh or session termination.
- **Maintaining a lightweight user experience:** The Playground is intended as a learning and experimentation tool, so execution features must be implemented without introducing heavy dependencies or performance overhead.

PROPOSED SOLUTION

To address this limitation, I propose extending the Template Playground to support execution of template logic through a trigger-based execution pipeline.

The solution integrates the existing Accord execution engine using TemplateArchiveProcessor.trigger(), enabling users to provide structured request inputs and receive corresponding outputs such as responses, events, and state updates.
This transforms the Playground from a static preview tool into an interactive environment for testing and understanding smart contract behavior.
The solution introduces:

- A request input panel for JSON-based data submission
- A Run Logic control to trigger execution
- A Run Logic control to trigger execution

This approach aligns with the existing Accord architecture and avoids reimplementing execution logic, ensuring consistency while keeping the Playground lightweight.

Addressing Key Challenges

The proposed solution addresses the identified challenges as follows:

- **Browser limitations:** By leveraging the existing in-browser execution pipeline, the solution avoids reliance on Node.js environments.
- **TypeScript execution:** The Playground reuses twoslash-based compilation and a JavaScript evaluator to safely execute contract logic in the browser.
- **Unsupported dependencies:** Execution errors caused by unsupported imports will be handled through clear error feedback in the UI.
- **Lack of persistence:** The system focuses on session-based simulation, aligning with the Playground's purpose as a learning environment.
- **Lightweight design:** By reusing the existing Accord execution engine instead of introducing new runtime systems, the solution maintains a lightweight architecture.

Implementation

To enable execution of template logic in the Template Playground, the existing rendering pipeline will be extended by integrating a logic execution layer alongside the current TemplateMark rendering flow.

The implementation will be performed directly within the playground architecture, primarily modifying the state management(store.ts) and UI components.

Current execution flow

```
rebuild(templateMarkdown, modelCto, data)
 → TemplateMarkTransformer.fromMarkdownTemplate()
  → TemplateMarkInterpreter.generate()
  → transform(..., "html")
  → agreementHtml (preview)
```

This flow generates an AgreementMark representation, transforms it into HTML, and renders it in the preview panel. No contract logic execution is performed in this pipeline.

Inside store.ts, the system only does

```
const result = await transform(
ciceroMarkJson,
"ciceromark_parsed",
["html"]
);
```

This confirms that the current system does not include an execution engine.

❖ Add Execution Pipeline

Introduce a parallel execution flow inside store.ts.

Execution Integration (Core Implementation)

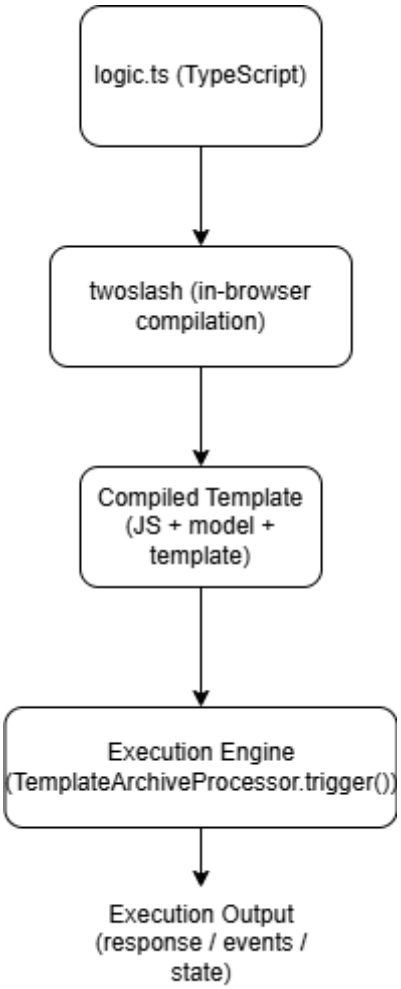
A parallel execution workflow will be introduced within [store.ts]() to enable contract execution using the existing Accord engine.

```
executeLogic: async () => {
  const { templateMarkdown, modelCto, requestData } = get();

  // Step 1: Load template
  const template = await Template.fromDirectory(/* built template */);

  // Step 2: Create processor
  const processor = new TemplateArchiveProcessor(template);

  // Step 3: Parse request input
  const request = JSON.parse(requestData);

  // Step 4: Execute logic
  const response = await processor.trigger(request);

  // Step 5: Store result
  set(() => ({
    executionResult: JSON.stringify(response, null, 2)
  }));
}
```

The Playground already supports in-browser TypeScript compilation using the twoslash library, which compiles contract logic (logic.ts) into JavaScript.
The execution engine internally compiles and evaluates contract logic using in-browser TypeScript compilation (twoslash) and a JavaScript evaluator, which executes the compiled logic in a sandboxed environment.
The proposed solution builds on this existing pipeline, where the compiled logic is executed using TemplateArchiveProcessor.trigger().

This approach ensures that execution is handled by the existing Accord engine without introducing a new execution mechanism, keeping the Playground lightweight and consistent with the overall architecture.

Figure: Contract execution flow in Template Playground using in-browser TypeScript compilation and Accord execution engine



Error Handling Strategy

To ensure a robust user experience, error handling will be integrated into the execution workflow.

- **Execution Failures:** Errors occurring during trigger() execution will be captured and displayed clearly in the UI.

- **Unsupported Dependencies:** Templates that rely on unsupported imports or Node.js-specific modules may fail in the browser environment. Such cases will be detected and reported with meaningful error messages.
- **Invalid Request Input:** Incorrect or malformed JSON input will be validated before execution, with user-friendly feedback provided.
- **Compilation Errors:** Errors during TypeScript compilation (via twoslash) will be surfaced to users to help debug logic issues.

Execution Engine Role

The playground does not implement contract execution internally. Instead, it leverages the Accord execution engine through:

`TemplateArchiveProcessor.trigger()`

When this method is invoked, the engine loads the compiled template archive, which includes the Concerto model, TemplateMark template, and associated logic.ts file. It then executes the contract logic defined within the template using the provided request input and produces a structured output, such as responses, events or state updates.

By leveraging the existing execution engine rather than reimplementing logic within the playground, this approach ensures consistency with overall Accord architecture while keeping the playground lightweight and focused on user interaction and visualization.

State Management Changes

To support contract execution, the existing state management in store.ts will be extended to include additional fields for handling execution inputs and outputs. Specifically, a new `requestData` field will capture the user provided request JSON, while an `executionResult` field will store the response generated by the execution engine.

These additions will follow the same state management patterns already used in the playground, ensuring consistency with existing fields such as `data` and `agreementHTML`. This approach allows seamless integration of execution functionality without disrupting the current rendering workflow.

❖ UI Integration

To enable interaction with execution pipeline, the Template playground interface will be extended with additional UI components that allow users to provide input and execution result

Request Input Panel

A dedicated Request Input Panel will be added to the Playground to capture request data required for contract execution. Users will provide input in JSON format, which will be passed to the execution engine.

The panel will mirror the existing Agreement Data editor, ensuring consistency in layout and interaction.

Fig: Existing Agreement Data (JSON Data) editor in Template Playground

The Request Input Panel will reuse the same editor structure, allowing users to input request JSON in a familiar interface. The input will be managed through the state layer and used during execution.

Run Logic Control

A Run Logic control will be added to allow users to explicitly trigger contract execution. It will be placed within the existing interface (toolbar or main container) to ensure easy access during the workflow.
When triggered, it will invoke the execution function (executeLogic) from the state layer and pass the user-provided request input to the execution engine.

This include:

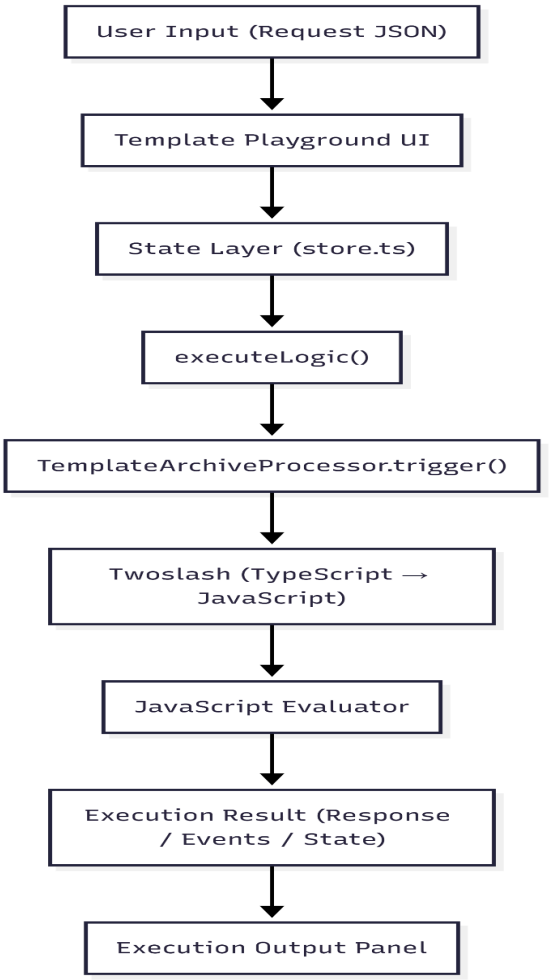
- ➔ Calls the execution workflow (executeLogic).
- ➔ Uses request input provided in the Request Input Panel.
- ➔ Triggers execution via the existing engine (trigger()).
- ➔ Displays results in the Execution Output Panel.

Execution Output Panel

- ➔ Add a dedicated panel to display execution results.
- ➔ Show output rendered by execution engine(trigger()).
- ➔ Display data in JSON format (response, state, event updates).
- ➔ Add a dedicated panel to display execution results.
- ➔ Keep format consistent with existing JSON editors.
- ➔ Allow users to quickly view and analyze execution results.

System Architecture & Data Flow

The extended architecture enables contract logic execution by routing user-provided request input through the state layer to the Accord execution engine. The logic is compiled in-browser using twoslash and executed via a JavaScript evaluator, with results displayed in the output panel.


User Input (Request JSON)
Template Playground UI
State Layer (store.ts)
executeLogic()
TemplateArchiveProcessor.trigger()
Twoslash (TypeScript → JavaScript)
JavaScript Evaluator
Execution Result (Response / Events / State)
Execution Output Panel

TIMELINE

Period	Task
Pre GSOC	➢ Set up development environment and run Template Playground locally ➢ Explore codebase and understand rendering + execution flow
COMMUNITY BONDING PERIOD	
May 1 - May 24	➢ Discuss with mentors about the best possible implementations of the above approach. ➢ Explore required tools and libraries . ➢ Explore TemplateArchiveProcessor.trigger() and twoslash flow
PHASE 1 **(Execution Integration)** **Milestone: Full execution pipeline working**	
Week 01(May 25 - May 31)	➢ Set up execution flow structure in store.ts ➢ Set up execution flow structure in store.ts.
Week 02 (June 1 – June 7)	➢ Integrate TemplateArchiveProcessor.trigger() ➢ Prepare request JSON handling
Week 03 (June 8 – June 14)	➢ Connect request input with execution pipeline ➢ Ensure data flows correctly to execution engine
Week 04 (June 15 - June 21)	➢ Implement twoslash-based TypeScript → JavaScript compilation flow ➢ Verify logic compilation inside browser
Week 05 (June 22 – June 28)	➢ Integrate JavaScriptEvaluator for executing compiled logic

	➢ Validate execution results (response/events/state)
Week 06 (June 29 – July 6)	➢ Store execution results in state (executionResult) ➢ End-to-end testing with sample templates
Midterm evaluation (July 6 - July 10)	➢ Demonstrate working execution pipeline ➢ Fix issues based on mentor feedback ➢ Stabilize core functionality
PHASE 2 **UI Integration & Refinement**	
Week 07 (July 6 – July 13)	➢ Add Request Input Panel (JSON editor) ➢ Connect UI with state (requestData)
Week 08 (July 14 – July 20)	➢ Implement Run Logic button ➢ Trigger execution from UI
Week 09 (July 21 – July 27)	➢ Build Execution Output Panel ➢ Display formatted execution results
Week 10 (July 28 – August 3)	➢ Add error handling (invalid JSON, runtime errors) ➢ Handle unsupported imports gracefully
Week 11 - week 12 (August 4 – August 16)	➢ Improve UI/UX consistency ➢ Optimize performance and responsiveness ➢ Final integration testing ➢ Code cleanup and documentation
Final Evaluation (August 17 – August 24)	➢ Final bug fixes ➢ Prepare demo and documentation ➢ Submit the final evaluation

MY COMMITMENTS

I do not have any prior commitments during the GSoC period and will be fully available to dedicate my time to this project. As my postgraduate program concludes on May 1, I will be able to commit approximately 40 hours per week throughout the duration of the program.

I will actively engage with the community, maintain regular communication with my mentors, and provide consistent updates on my progress.

EXPECTATIONS FROM MENTORS

- Guidance in choosing the most appropriate implementation approach when multiple solutions are possible.
- Support in understanding complex parts of the codebase when required.
- Regular feedback on progress, design decisions, and implemented features.

WHY ME

I have experience in web development and building tools such as Chrome extensions, which led me to explore open source. Since December 2025, I have been actively exploring the Accord Project ecosystem, particularly the Template Playground. I also built a prototype to simulate the execution flow, which helped me understand the gap between the current Playground and the proposed functionality.

I believe that, even in today's AI-driven environment, real value comes from deeply understanding systems and solving problems step by step. I focus on analyzing how each part works before implementing solutions and maintain clear communication with mentors and maintainers.This project also interests me because it can make smart contract development more accessible to non-technical users. Contributing to improving such a learning tool would be meaningful, and I am excited to be a part of it.

POST GSOC

I intend to continue contributing to the Accord Project after the GSoC period by improving the Template Playground and related features. I will actively work on enhancing editor capabilities, fixing issues, and contributing wherever possible.

It would be a great opportunity to remain part of the Accord Project community and contribute towards making its tools more accessible and useful for users.